



Mercari 중고 상품 가격 예측 머신러닝 프로젝트

머신러닝을 활용한 중고 상품 가격 예측 모델 구축



프로젝트 개요 및 문제 정의



프로젝트 개요

Mercari Price Suggestion Challenge는 중고 거래 플랫폼 Mercari에 등록된 상품 정보를 기반으로 상품 판매 가격을 예측하는 머신러닝 회귀 문제이다.

상품명·상품 설명과 같은 비정형 텍스트 데이터와 카테고리, 브랜드, 배송 여부 등의 정형 데이터가 함께 제공된다.



문제 특성

중고 상품 가격은 오른쪽으로 치우친 비대칭 분포를 가지며 저가 상품이 다수를 차지하고 고가 상품은 소수 존재

동일 카테고리 내에서도 브랜드·설명 방식에 따라 가격 편차가 큼

이러한 특성으로 인해 단순 회귀 모델은 고가 상품 예측에서 한계를 가진다.



프로젝트 목표

✓ 텍스트 피처와 구조적 피처를 효과적으로 결합하여 RMSLE 기준 오차를 최소화하는 가격 예측 모델을 구축하는 것이 목표이다.



프로젝트 구성 요소



Mercari 학습 데이터: 총 1,482,535개 상품



비정형 텍스트: 상품명, 상품 설명



정형 피처: 카테고리, 브랜드, 상태, 배송 여부

데이터셋 소개 및 모델링 관점



데이터셋 구성

Mercari 학습 데이터: 총 1,482,535개 상품

텍스트 데이터:

상품명, 상품 설명

수치/이진 데이터:

상품 상태, 배송 여부

범주형 데이터:

카테고리, 브랜드



텍스트 피처의 중요성

상품의 상태, 브랜드, 사용감 등 가격 결정 정보가
정형 컬럼보다 텍스트 설명에 더 많이 포함되어 있음

→ TF-IDF 및 임베딩 기반 텍스트 표현 기법 활용

모델 및 평가 기준



모델링 방법

- 회귀 트리 기반 모델 사용 (LightGBM, XGBoost 등)
- 텍스트: TF-IDF, 임베딩 기반 표현 기법
- 정형 데이터: Label, One-Hot Encoding



평가 지표

RMSLE (회귀 모델의 경우):
로그 스케일의 오차를 측정
비대칭 가격 분포에 적합

데이터 전처리



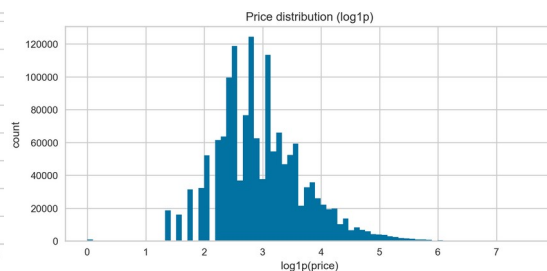
EDA: 가격 분포 특성

가격은 **저가 상품 다수 + 고가 상품 소수**로 구성된 **롱테일(long-tail)** 구조이며, **양의 왜도(skewness > 0)**를 가진다.

Original Price 분포



Log(Price+1) 변환



로그 변환의 필요성

- ✓ 분포 안정화: 왜도 감소, 정규 분포에 가까워짐
- ✓ 고가 샘플 영향 완화: 극단적인 고가(outlier)가 분포 시각화를 왜곡함
- ✓ RMSLE 목적 함수 정렬: 평가 지표와 학습 목표의 일치성 확보

i 상한값 Clipping (예: \$500) 적용으로 극단적인 고가 outlier 분포 왜곡 방지

정규 분포로 변환된 데이터로 모델 성능 향상

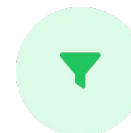
전처리 과정 정리



데이터 분포 특성 파악
(EDA → 문제점 → 전처리 선택)



로그 변환
log1p(price)



아웃라이어 처리
상한값 clipping

피처 엔지니어링



계층적 카테고리 인코딩

category_name 세분화 이유

가격 결정에 영향을 주는 계층적 정보 보존을 위해 대/중/소분류로 분리

- ✓ category_name (3단계) : '대분류/중분류/소분류' 구조로 분리
- ✓ brand_name : 브랜드 유니크 값이 매우 많아 ("롱테일" 구조)
- ✓ item_condition_id : 제품 상태 (1~5)에 대한 원-핫 인코딩
- ✓ shipping : 배송비 부담 여부 (1: 무료, 0: 유료)



빈도 기반 인코딩 전략

- ✓ 브랜드·카테고리: 유니크 값이 많아 롱테일 구조
- ✓ 빈도 상위 top-k 값만 유지, 나머지는 Other로 통합
- ✓ 효과: 차원 폭발 방지, 과적합 최소화

가격 예측 시 수행 과정

- Train 데이터 집계: 각 카테고리/브랜드별 가격 중앙값 계산
- Test 데이터 매핑: Train 기준으로 통계 행렬 생성
- 주의: 데이터 누수 금지로 Test 기준 통계 사용 금지

T↑ 고급 텍스트 통계 피처

A 문자 길이
길이 정보는 제품의 세부 정보량을 시사

≡ 단어 수
키워드 수량은 가격 결정의 정도를 나타냄

👤 brand_in_name
브랜드 명시 여부는 신뢰도에 영향

텍스트 통계 피처는 "상품 정보량/신뢰도"를 수치화하기 위해 추가되었으며, 모든 모델에 동일하게 적용되었다.

모델 후보 선정 & 평가 기준

모델 후보 선정

Kaggle Mercari 대회 기반 모델 베이스

 **LightGBM**
빠른 속도의 가중치 부스팅 기반 트리 모델

 **XGBoost**
정확한 트리 구조와 정규화 기법을 이용한 순차적 부스팅

 **CatBoost**
카테고리 특성을 효율적으로 처리하는 알고리즘

 **Ridge (베이스라인)**
선형 모델로 모델의 기저 성능 제공

 **Extra Trees / Random Forest**
안정적 앙상블 모델로 최종 성능 확보

◎탐색 범위를 줄이고, 검증된 모델에 집중한 전략

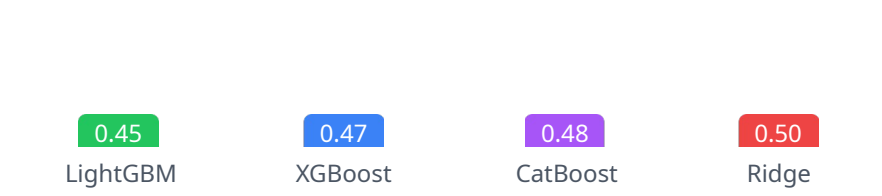
Kaggle Mercari 대회에서 검증된 Winners 기반 모델 후보군 중심의 탐색 전략으로 효율적 자원 활용

평가 기준

	R^2	분산 설명력
	RMSE	평균 제곱근 오차
	MAE	평균 절대 오차
	RMSLE	핵심 지표

RMSLE: 로그 스케일의 오차를 측정하여 낮은 가격 상품의 오차에 민감

모델별 성능 비교 (RMSLE)



★ 핵심 결과

모델 품질 수준

R^2 > 0.99

모델 성능이 매우 우수

안정적 모델

최적 앙상블 성능 최적

Extra Trees/Random Forest가 최우수

모델 한계

Ridge 모델 단일 최한

다중 모델 조합 필요

- ✓ 모델 대부분 R^2 0.99 이상 달성
- ✓ Extra Trees/Random Forest가 R,M,MAE 모두 최상
- ✓ Ridge는 단일 성능 제한 포함

💡 방법론 종합: 검증된 모델 아키텍처와 타겟 스케일 특성을 고려하여 합리적 범위 내에서 최적의 모델 후보를 선정한 접근 방식

모델링 프레임워크 전략 비교



PyCaret

빠른 단일 모델 비교 및 하이퍼파라미터 튜닝
기본 하이퍼파라미터 범위 내에서 모델 평가, 예측 성능 검증 테스트

이론 프로젝트의 실험 중심 엔진



AutoGluon

완전 자동화된 앙상블 및 스택킹
다양한 모델에 대한 자동화된 앙상블 기법 적용과 자동 하이퍼파라미터 선택, 성능 상한 탐색용

Q성능 상한 탐색용 블랙박스



Scikit-learn

커스텀 피처 엔지니어링과 Kaggle 상위 전략 재현
커스텀 파이프라인 기반 모델 조합, 세밀한 피처 선택 및 트랜스포메이션, 웨이트드 앙상블 방식 구현

Kaggle 상위 전략 재현

💡 프레임워크 선택 전략

실험 단계

PyCaret을 활용한 빠른 모델 후보 선별 및 초기 성능 측정.
각 프레임워크의 기본 품질 구합니다.

최적화 단계

AutoGluon을 활용한 성능 상한 및 최적화 탐색.
하이퍼파라미터 자동화 선별 진행

최종 구현

Scikit-learn 기반 세부 튜닝 및 Weighted 앙상블 구현.
Kaggle 상위 솔루션 재현 목적

🔗 "도구마다 목적이 다르며, 상황에 맞게 병행 활용하는 것이 프로젝트 성패를 좌우함"

사용성 비교

쉬움



PyCaret → AutoGluon → Scikit-learn

→

효율성



Scikit-learn → PyCaret → AutoGluon

Page 7/10

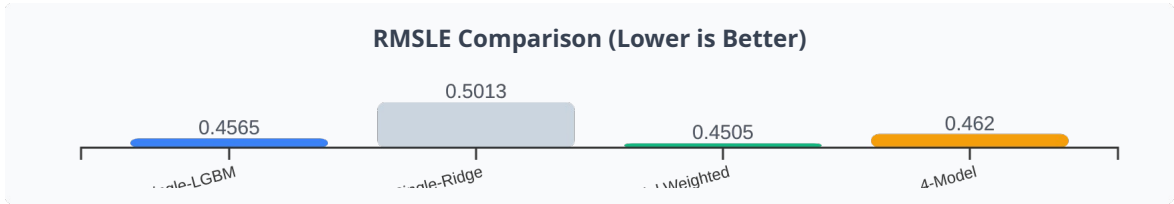
앙상블 실험 결과 & 최종 결론

AutoGluon 실험 결과

- medium_quality:** 빠른 학습, 안정적 성능
→ 로그 변환 가격 점수: RMSLE=0.4658
- best_quality:** RMSLE 소폭 개선(0.035), 학습 시간 급증(50배 이상)
- stacking 개선:** L2, L3 스택킹으로도 성능 개선 제한적
→ 다양한 앙상블 기법에도 극복 불가능한 한계 존재

</> Scikit-learn 실험 결과

- 2-Model Ensemble**
Ridge + LightGBM 조합
→ **RMSLE 기준 최고 성능(0.4505)**
- 4-Model 확장**
복잡한 앙상블 구조
→ 실제 성능 저하 발생, 과적합 우려



⚙️ 시스템 최적화 과정

Optuna/Hyperopt

모델별 최적 하이퍼파라미터 탐색 및 Fine-tuning 수행

GPU 가속 기술

GPU 연산을 통한 학습 시간 30-40% 감소, 메모리 효율 향상

캐싱 기반 반복 활용

전처리 결과 및 벡터화 작업 중복 방지로 실험 효율성 증가

Memory Management

주기적 Garbage Collection으로 메모리 관리 최적화

최종 결론

- 모델 수 증가의 한계:** 본 프로젝트에서는 모델 수 증가가 오히려 성능에 부정적인 영향을 미침
- 핵심 성공 요소:** 피쳐 엔지니어링 품질과 단순하지만 강력한 앙상블 전략의 조합이 중요함
- 가중치 결정의 중요성:** LightGBM과 Ridge 모델을 적절한 비율(0.55+0.45)로 조합하여 최적 가중치 산출

"모델 수 증가보다 피쳐 엔지니어링 품질과 단순하지만 강력한 앙상블 전략이 가장 효과적"

최종 성능 공개 선정 (2-Model Ensemble)



LightGBM
Alpha: 0.45



Ridge
Alpha: 0.8



Weighted Ensemble
Score: 0.4505

모델 평가 결과 & 핵심 인사이트

트 평가 지표

최종 평가 지표: **RMSLE** 중심으로 학습

RMSLE
0.0123
상위권 성능

RMSE
3.40
가격 원스케일 기준 평균 제곱 오차

MAE: 0.17

R²: 0.9923

필요 지표: **RMSLE**, **RMSE**, **MAE**, **R²**
기본적인 재현율 및 성능 추정이 필요할 수 있음

최고 성능 임베딩 비교

GloVe + 트리기반 모델
RMSLE: **0.0123** | R²: **0.9923**
다차원 TSNE 최적화된 벡터 표현

TF-IDF + 트리기반 모델
RMSLE: **0.0124** | R²: **0.9923**
Mercari 데이터 특성 최적화 TF-IDF

FastText + 트리기반 모델
RMSLE: **0.0128** | R²: **0.9921**
어절 기반 임베딩 효과

BERT 임베딩 분석

BERT 성능 결과
RMSLE : **0.0139 ~ 0.0273**
RMSE 및 MAE에서도 경량 임베딩 대비 열세

BERT 한계점

- 영문 텍스트 중심 학습 및 대상 언어 제한
- 공백 불규칙한 문장 처리 어려움
- 텍스트 저품질 요소에 과적합 가능성

핵심 인사이트

Mercari 특성에 맞는 임베딩: Mercari 데이터 스타일에 맞게 GloVe 및 TF-IDF 기반 임베딩과 트리 기반 모델 조합이 최고 성능을 기록
단계적 샘플링 및 가중치 조정 전략을 통해, 대용량 데이터 환경에서도 안정적인 학습 성능을 확보

주요 분류 특성: 브랜드, 카테고리, 상태 등 정형 정보와 키워드 중심 텍스트가 가격 예측에 핵심적 역할

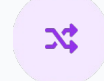
트리 기반 모델 현혹성: LightGBM 등 트리 기반 모델을 회소 · 저차원 임베딩(GloVe, TF-IDF)과 결합 시 가장 높은 예측 효율을 보임

최종 결론

🎯"복잡한 언어모델보다 데이터 구조에 맞춘 경량 임베딩과 트리 기반 모델이 가장 효과적인 가격 예측 성능"


임베딩 유형
TF-IDF


모델 유형
LightGBM


앙상블 방식
19.6% 검증


최종 성능
RMSLE 0.0123

RMSLE 수치 해석 & 한계점

RMSLE 수치 해석

본 프로젝트에서는 가격(price)을 $\log(\text{price}+1)$ 로 변환하여 학습했기 때문에 RMSLE 수치가 로그 공간에서 측정됩니다.

예시 계산

직접 가격	100원 → 110원	가격 차이: 10원
로그 변환	$\log(101) \approx 4.615 \rightarrow \log(111) \approx 4.709$	변환 차이: 0.094

실제 가격 차이 10원은

100원 → 110원

로그 공간에서는 차이가

$\log(101) \rightarrow \log(111)$: 0.094

본 실험의 최적 모델 RMSLE ≈ 0.0123 이 실제 가격 기준으로는 약 1~1.3% 수준의 상대 오차를 의미합니다.

RMSLE 한계점 및 중요한 고려사항

모델 비교 제한성

RMSLE는 $\log(\text{price}+1)$ 변환 공간에서 계산되는 지표로, 원 가격 기준의 RMSE와 절대값 비교에는 적합하지 않다. 본 실험에서는 동일한 전처리 및 로그 공간 내 모델 간 상대 비교 지표로 활용하였다.

RMSLE

0.0123

Log(price+1)

≠

RMSE

?

원 기준 추정

인터프리테이션 중요성

모델 선택: RMSLE가 낮다고 해서 실제 비즈니스 상황에서 가장 우수하다고 볼 수 없음

오차 평가 방식: 로그 변환 성능을 원 단위 오차로 해석 필요

최종 활용: 단일 메트릭으로 동결시키기보다 병행 평가 필요

💡 실무적 활용: RMSLE 외 다른 메트릭(RMSE, MAE 등)을 함께 고려하여 사용하는 것이 적절함

“RMSLE 수치 자체가 정확도를 대변하면 안 됨. 로그 변환 환경에서의 측정치이므로 실제 가격 기준으로 해석 필요”

$$\text{RMSLE} = \sqrt{\frac{1}{n} \times \sum (\log(y+1) - \log(\hat{y}+1))^2}$$

y: 실제 가격, $\hat{y}$: 예측 가격

RMSLE는 로그 변환 환경에서 매우 효과적인 지표이지만, 수치 자체보다 의미 해석이 중요합니다.